

فصل ۶

ابزار همگامی

پردازه‌ای همکار نامیده می‌شود که بتواند روی اجرای سایر پردازه‌های سیستم تأثیر گذاشته یا از اجرای آنها متأثر شود. پردازه‌های همکار می‌توانند یک فضای آدرس منطقی (یعنی، هر دوی کد و داده‌ها) را مستقیماً به اشتراک گذاشته و یا تنها اجازه دارند داده‌ها را از طریق حافظه‌ی مشترک یا عبور پیغام به‌اشتراک بگیرند. با این وجود، دسترسی همروند به داده‌های مشترک می‌تواند منجر به ناسازگاری داده‌ها شود. در این فصل، به بحث راهکارهای مختلفی می‌پردازیم که اجرای مرتب پردازه‌های همکار را در یک فضای آدرس منطقی مشترک اطمینان می‌دهند، طوری که سازگاری داده‌ها هم حفظ می‌شود.

اهداف فصل

- تبیین مسأله‌ی بخش بحرانی و نشان دادن یک شرایط مسابقه.
- نشان دادن راه‌حل‌های سخت‌افزاری برای مسأله‌ی بخش بحرانی با استفاده از سدهای حافظه، عملیات compare-and-swap، و متغیرهای یکپارچه.
- نمایش چگونگی استفاده از قفل‌های انحصار متقابل، سمافورها، مانیتورها، و متغیرهای شرط برای حل مسئله‌ی بخش بحرانی.
- ارزیابی ابزاری که مسئله‌ی بخش بحرانی را در شرایط رقابت پایین، متوسط، و بالا حل می‌کنند.

۶-۱ پس‌زمینه

قبلاً دیده‌ایم که پردازه‌ها می‌توانند به صورت همروند یا موازی اجرا شوند. بخش ۳-۲-۲، نقش زمانبندی پردازه را معرفی و عنوان کرد که زمانبند پردازه چگونه اجرای همروند را با انتقال سریع میان پردازه‌ها فراهم می‌کند. این بدان معنی است که یک پردازه ممکن است قبل از زمانبندی پردازه‌ی دیگر فقط اجرای خود را به صورت جزئی تکمیل نماید. در حقیقت، یک پردازه امکان دارد که در هر نقطه از جریان دستورالعمل خود متوقف شده و هسته‌ی پردازشی به اجرای دستورالعمل‌های پردازه‌ی دیگر گمارده شود. به علاوه، بخش ۴-۲ اجرای موازی را مطرح کرد که در آن، دو جریان دستورالعمل (معرف پردازه‌های متفاوت) به صورت همزمان بر روی هسته‌های پردازشی مجزا اجرا دارند. در این فصل، توضیح می‌دهیم که اجرای همروند یا موازی چگونه می‌تواند در ملاحظات مشمول جامعیت داده‌های مشترک پردازه‌های متعدد سهیم

باشد.

بیایید با یک مثال، چگونگی وقوع این موضوع را ببینیم. در فصل ۳، مدلی از یک سیستم را شامل پردازه‌ها یا نخ‌های ترتیبی همکار توسعه دادیم که همگی اجرای ناهمگام داشتند و امکان اشتراک داده‌ها نیز وجود داشت. این مدل را با استفاده از مسأله‌ی تولیدکننده–مصرف‌کننده نشان دادیم که یک سبک نمونه‌ای از کارکردهای فراوان سیستم عامل است. به‌طور خاص، در بخش ۳–۵، چگونگی استفاده از بافر محدود را مطرح کردیم که امکان اشتراک حافظه را برای پردازه‌ها فراهم می‌کند.

حال به ملاحظاتی که با بافر محدود داشتیم بر می‌گردیم. همان‌گونه که نشان دادیم، راه‌حل ما همزمان، حداکثر BUFFER_SIZE – 1 قلم‌داده را در بافر نگه می‌دارد. فرض کنید بخواهیم الگوریتم را اصلاح کنیم تا این نقص برطرف شود. یک روش این است که یک متغیر صحیح count با مقدار اولیه‌ی ۰ اضافه کنیم. هر بار که قلم‌داده‌ی جدیدی به بافر اضافه می‌شود، count یک واحد افزایش یابد و هر زمان که یک قلم‌داده از بافر برداشته می‌شود، count یک واحد کاهش می‌یابد. کد پردازه‌ی تولیدکننده می‌تواند به صورت زیر اصلاح شود:

```
while (true)
{
        /* produce an item in next_produced */
        while (count == BUFFER_SIZE)
                ; /* do nothing */
        buffer[in] = next_produced;
        in = (in + 1) % BUFFER_SIZE;
        count++;
}
```

کد پردازه‌ی مصرف‌کننده هم می‌تواند به صورت زیر اصلاح شود:

```
while (true)
{
        while (count == 0); /* do nothing */
        next_consumed = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
        count--;
        /* consume the item in next_consumed */
}
```

اگرچه روال‌های تولیدکننده و مصرف‌کننده که در بالا نشان داده می‌شوند، به صورت مجزا درست هستند، اما موقعی که اجرای همروند دارند ممکن است عملکرد درستی نداشته باشند. برای نمایشی از این مطلب، فرض کنید مقدار فعلی متغیر count، برابر ۵ است و پردازه‌های تولیدکننده و مصرف‌کننده دستورات "count++" و "count--" را به طور همروند اجرا می‌کنند. به دنبال اجرای این دو دستور، متغیر count می‌تواند مقدار ۴، ۵ یا ۶ را داشته باشد! حال آنکه تنها مقدار درست متغیر count برابر مقدار ۵ است که با اجرای مجزای تولیدکننده و مصرف‌کننده حاصل می‌شود.

می‌توانیم امکان نادرستی مقدار count را به صورت زیر نشان دهیم. توجه کنید که دستور "count++" ممکن است در زبان ماشین (روی ماشین نوعی) به صورت زیر پیاده‌سازی شود:

$$register_1 = \mathbf{count}$$
$$register_1 = register_1 + 1$$
$$\mathbf{count} = register_1$$

که در آن، $register_1$ یکی از ثبات‌های محلی پردازنده است. به طور مشابه، دستور "count--" به صورت زیر پیاده‌سازی می‌شود:

$$register_2 = \mathbf{count}$$
$$register_2 = register_2 - 1$$
$$\mathbf{count} = register_2$$

که باز هم $register_2$ یک ثبات محلی پردازنده است. هرچند که $register_1$ و $register_2$ ممکن است ثبات فیزیکی یکسانی باشند، به یاد داشته باشید که محتوای این ثبات، توسط راه‌انداز وقفه، ذخیره و بازیابی خواهد شد (بخش ۱-۲-۳).

اجرای همروند دستورات "count++" و "count--" با یک اجرای ترتیبی معادل است که در آن، دستورات سطح پایین‌تری که پیشتر ارائه شدند با ترتیب دلخواهی (اما با حفظ ترتیب میان هر یک از دستورات سطح بالا) یک در میان قرار می‌گیرند. یکی از ترتیب‌های یک در میان این چنینی به صورت زیر است:

T_0: تولیدکننده $\mathbf{count}$ = $register_1$ را اجرا می‌کند $[register_1 = 5]$

T_1: تولیدکننده $register_1 = register_1 + 1$ را اجرا می‌کند $[register_1 = 6]$

T_2: مصرف‌کننده $\mathbf{count}$ = $register_2$ را اجرا می‌کند $[register_2 = 5]$

T_3: مصرف‌کننده $register_2 = register_2 - 1$ را اجرا می‌کند $[register_2 = 4]$

T_4: تولیدکننده $\mathbf{count}$ = $register_1$ را اجرا می‌کند $[count = 6]$

T_5: مصرف‌کننده $\mathbf{count}$ = $register_2$ را اجرا می‌کند $[\mathbf{count = 4}]$

توجه کنید که به حالت نادرست "counter == 4" می‌رسیم که نشانگر وجود چهار بافر پر می‌باشد، حال آن که تعداد بافرهای پر واقعی پنج است. اگر ترتیب دستورات T_4 و T_5 را جابه‌جا کنیم، به حالت نادرست "counter == 6" خواهیم رسید.

دلیل این که به این حالت نادرست می‌رسیم، این است که به هر دو پردازه اجازه می‌دهیم، متغیر count را به طور همروند تغییر بدهند. چنین شرایطی که چندین پردازه داده‌های یکسانی را به طور همروند دسترسی و دستکاری می‌نمایند و حاصل اجرای آنها به ترتیب خاصی از دسترسی‌ها بستگی دارد، **شرط مسابقه**[1] نامیده می‌شود. برای مقابله با شرط مسابقه‌ی بالایی لازم است تضمین کنیم که متغیر count در هر لحظه تنها می‌تواند توسط یک پردازه دستکاری شود. برای حصول چنین تضمینی لازم است که پردازه‌ها به نحوی با یکدیگر همگام شوند.

وضعیت‌هایی مشابه آنچه که دیدید در سیستم‌عامل‌ها به کرات با دستکاری منابع توسط بخش‌های متفاوت سیستم پیش می‌آیند. به علاوه، همان‌گونه که در فصول قبلی تأکید کرده‌ایم، اهمیت سیستم‌های چندهسته‌ای، اصرار زیادی بر تولید برنامه‌های کاربردی چندنخی دارد. در این

1. Race condition

قبیل برنامه‌ها، چندین نخ—که به احتمال زیاد داده‌های مشترکی هم دارند—به صورت موازی بر روی هسته‌های پردازشی متفاوت اجرا می شوند. واضح است که می‌خواهیم تغییرات حاصل از این اعمال با همدیگر تداخلی نداشته باشند. به خاطر اهمیت این موضوع، بخش اعظمی از این فصل را به **همگامی پردازه** و **هماهنگی** مابین پردازه‌های همکار اختصاص می‌دهیم.

۶-۲ مسأله‌ی بخش بحرانی

موضوع همگامی پردازه را با بحث مسأله‌ی بخش بحرانی آغاز می‌کنیم. سیستمی شامل n پردازه‌ی $\{P_0, P_1, \ldots, P_{n-1}\}$ را در نظر بگیرید. هر پردازه یک قطعه کد به نام **بخش بحرانی**[1] دارد که در آن، پردازه ممکن است داده‌هایی را که با حداقل یک پردازه‌ی دیگر مشترک است، دسترسی کرده—و به‌روز—کند. ویژگی مهمی که در این سیستم وجود دارد این است که، زمانی که یک پردازه در بخش بحرانی خود اجرا می‌شود، هیچ پردازه‌ی دیگری اجازه ندارد که در بخش بحرانی خود باشد. یعنی، هیچ دو پردازه‌ای را نمی‌توان یافت که همزمان بخش بحرانی خود را اجرا نمایند. *مسأله‌ی بخش بحرانی*، طراحی پروتکلی است که پردازه‌ها بتوانند با استفاده از آن فعالیت خودشان را چنان هماهنگ کنند که داده‌ها را با همکاری به‌اشتراک بگذارند. هر پردازه بایستی برای ورود به بخش بحرانی اجازه بگیرد. بخش کدی که این درخواست را پیاده‌سازی می‌کند، **بخش ورود** نام دارد. به دنبال بخش بحرانی می‌تواند، **بخش خروج** قرار بگیرد. کدی که باقی می‌ماند، **بخش باقیمانده** است. ساختار کلی یک پردازه‌ی نوعی در شکل ۶-۱ نشان داده می‌شود. کد بخش‌های ورود و خروج با جعبه مستطیل نشان داده شده‌اند تا اهمیت این قطعات کد بارزتر شود.

هر راه‌حل مسأله‌ی بخش بحرانی بایستی نیازهای سه‌گانه‌ی زیر را ارضا نماید:

۱- **انحصار متقابل**[2]: اگر پردازه‌ی P_i در بخش بحرانی خود اجرا دارد، در این صورت هیچ یک از پردازه‌های دیگر نمی‌توانند در حال اجرای بخش بحرانی خود باشند.

۲- **پیشروی**: اگر هیچ پردازه‌ای در حال اجرای بخش بحرانی خود نباشد و برخی پردازه‌ها خواهان ورود به بخش بحرانی خود باشند، در این صورت فقط **پردازه‌هایی** می‌توانند در رابطه با انتخاب پردازه‌ی نوبت بعدی ورود به بخش بحرانی تصمیم بگیرند که در بخش باقیمانده‌ی خود اجرا ندارند و این انتخاب نمی‌تواند به طور نامعینی به تعویق بیفتد.

۳- **انتظار محدود**[3]: بعد از انجام درخواست یک پردازه جهت ورود به بخش بحرانی خود و پیش از اعطای درخواستش، بر روی تعداد دفعاتی که سایر پردازه‌ها می‌توانند وارد بخش بحرانی خود شوند یک کران یا محدودیت وجود دارد.

فرض ما بر این است که هر پردازه با یک سرعت غیرصفر اجرا می‌شود. با این وجود، هیچ فرضی در رابطه با سرعت نسبی n پردازه نداریم.

1. Critical section
2. Mutual exclusion
3. Bounded waiting

در هر نقطه از زمان مشخص، پردازه‌های زیادی ممکن است با مد هسته در سیستم‌عامل فعال باشند. در نتیجه، کدی که سیستم‌عامل را پیاده‌سازی می‌کند (**کد هسته**) مستعد چندین شرط مسابقه است. برای مثال، ساختمان‌داده‌ای از هسته را در نظر بگیرید که لیست تمامی فایل‌های باز سیستم را نگه می‌دارد. این لیست بایستی با باز یا بسته‌شدن یک فایل جدید تغییر یابد (اضافه‌کردن فایل به لیست یا حذف فایل از لیست). اگر دو پردازه همزمان فایل‌هایی را باز کرده باشند، به روز رسانی‌های مجزای این لیست می‌تواند یک شرط مسابقه به وجود آورد. ساختمان‌داده‌های دیگری از هسته که در معرض شرایط مسابقه هستند عبارتند از: ساختارهای نگهداری تخصیص حافظه، نگهداری لیست پردازه‌ها و رسیدگی به وقفه. این مسؤولیت توسعه‌دهندگان هسته است تا اطمینان بدهند که یک سیستم‌عامل عاری از شرایط مسابقه‌ی این مدلی است.

```
while (true) {

        entry section

                critical section

        exit section

        remainder section

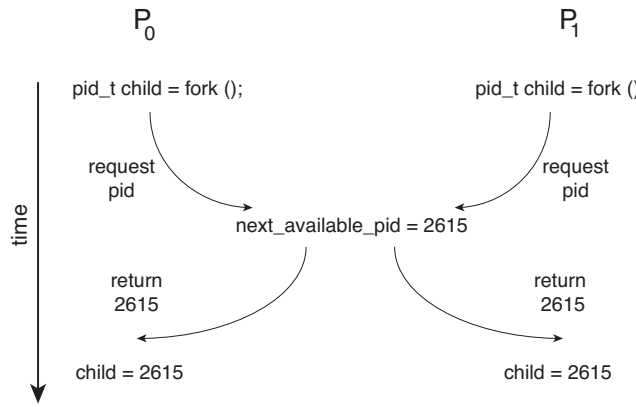
}
```

شکل ۶-۱ ساختار کلی پردازه‌ی نوعی P_i.

مثال دیگری در شکل ۶-۲ نشان داده می‌شود. در اینجا، دو پردازه، P_0 و P_1 با استفاده از فراخوانی سیستمی ()fork پردازه‌های فرزند ایجاد می‌کنند. از بخش ۳-۳-۱ به‌خاطر بیاورید که ()fork شناسه‌ی پردازه‌ای را به‌تازگی ایجاد شده به پردازه‌ی والد بر می‌گرداند. در این مثال، یک شرایط مسابقه بر روی متغیر هسته‌ی next_available_pid وجود دارد که معرف مقدار شناسه‌ی بعدی در دسترس برای پردازه است. مگر این که انحصار متقابل فراهم شود، این امکان هست که شماره‌ی شناسه‌ی پردازه‌ی یکسانی به دو پردازه‌ی مجزا گمارده شود.

سایر ساختمان‌داده‌های هسته که مستعد شرایط مسابقه‌ی احتمالی هستند عبارتند از: ساختارهای حفظ حافظه‌ی تخصیص‌یافته، حفظ لیست پردازه‌ها، و رسیدگی به‌وقفه. مسئوولیت توسعه‌دهندگان هسته است تا مطمئن شوند که سیستم‌عامل عاری از شرایط مسابقه است.

مسئله‌ی بخش بحرانی می‌توانست به‌سادگی در یک محیط تک‌هسته‌ای حل شود اگر می‌توانستیم در حالی که یک متغیر مشترک تغییر داده می‌شد از وقوع وقفه‌ها پیشگیری کنیم. با این شیوه، می‌توانستیم اطمینان پیدا کنیم که توالی دستورالعمل‌های فعلی اجازه خواهد داشت بدون پس‌گیری به‌ترتیب اجرا شود. هیچ دستورالعمل دیگری اجرا نمی‌شد، لذا هیچ‌گونه تغییر ناخواسته‌ای بر روی متغیر مشترک انجام نمی‌گرفت.



شکل ۶-۲ شرایط مسابقه موقع انتساب یک pid.

متأسفانه، این راه‌حل با این شیوه در یک محیط چندپردازنده امکان‌پذیر نیست. غیرفعال‌کردن وقفه‌ها روی یک چندپردازنده می‌تواند وقت‌گیر باشد، از آنجایی که پیغام بایستی به‌تمامی پردازنده‌ها ارسال شود. این عبور پیغام، ورود به بخش بحرانی را به‌تأخیر انداخته، و کارآمدی سیستم کاهش پیدا می‌کند. همین‌طور، اثر آن را بر روی ساعت سیستم در نظر بگیرید در صورتی که قرار باشد ساعت با وقفه‌ها به‌روز نگه داشته شود.

دو رویکرد کلی برای رسیدگی به بخش‌های بحرانی در سیستم‌عامل استفاده می‌شوند که عبارتند از: **هسته‌های پس‌گرفتنی**[1] و **هسته‌های غیرقابل پس‌گیر**[2]. هسته‌ی پس‌گرفتنی، امکان پس‌گیری پردازه را در حالی‌که در مد هسته اجرا می‌شود فراهم می‌کند. در هسته‌ی غیرقابل پس‌گیر، امکان پس‌گیری پردازه‌ای که در مد هسته اجرا دارد، وجود ندارد؛ پردازه‌ای که در مد هسته اجرا می‌شود تا زمانی اجرا خواهد شد که از مد هسته خارج شود، مسدود شود و یا این که کنترل پردازنده را داوطلبانه پس دهد.

واضح است که هسته‌ی غیرقابل پس‌گیر به واقع بر روی ساختمان‌داده‌های هسته عاری از شرایط مسابقه می‌باشد، چرا که در هر لحظه فقط یک پردازه در هسته فعال است. این گفته در مورد هسته‌های پس‌گرفتنی صادق نیست، از این رو برای اطمینان از این که داده‌های مشترک هسته از شرایط مسابقه رها خواهند بود بایستی طراحی آنها به دقت صورت گیرد. هسته‌های پس‌گرفتنی مخصوصاً در طراحی برای معماری‌های SMP مشکل هستند، چون در این محیط‌ها امکان اجرای همزمان دو پردازه بر روی هسته‌های متفاوت پردازنده با مد هسته وجود دارد.

اکنون این سؤال مطرح می‌شود که چرا بایستی هسته‌ی پس‌گرفتنی به هسته‌ی غیرقابل پس‌گیر ارجحیت داشته باشد؟ هسته‌ی پس‌گرفتنی برای برنامه‌نویسی بلادرنگ مناسب‌تر است، چرا که به پردازه‌ی بلادرنگ اجازه می‌دهد تا پردازه‌ای را که در هسته اجرا می‌شود پس بگیرد. به علاوه، زمان پاسخ هسته‌ی پس‌گرفتنی بهتر خواهد بود، چون این خطر کمتر است که یک پردازه بتواند برای مدت طولانی دلخواهی در مد هسته اجرا شود پیش از آنکه پردازنده را برای

1. Preemptive kernels
2. Nonpreemptive kernels

پردازه‌های منتظر رها کند. (البته، کد هسته را می‌توان چنان طراحی کرد که رفتاری به این نحو نداشته و این خطر هم کمینه شود.) به علاوه، هسته‌ی پس‌گرفتنی برای برنامه‌نویسی بلادرنگ مناسب‌تر است، چرا که به پردازه‌ی بلادرنگ امکان می‌دهد تا پردازه‌ای را که در هسته اجرا دارد، پس بگیرد. در ادامه‌ی همین فصل، چگونگی مدیریت پس‌گیری در داخل هسته را در سیستم‌عامل‌های مختلف خواهیم دید.

۶-۳ راه‌حل پترسن

در ادامه، یک راه‌حل کلاسیک نرم‌افزاری، معروف به **راه‌حل پترسن**[1] را برای مسأله‌ی بخش بحرانی نشان می‌دهیم. به خاطر روش انجام دستورالعمل‌های زبان ماشین، نظیر load و store در معماری‌های مدرن کامپیوتر، هیچ تضمینی برای صحت کار راه‌حل پترسن در این نوع معماری‌ها وجود ندارد. با این وجود، راه‌حل را به این دلیل مطرح می‌کنیم که توصیف الگوریتمی خوبی برای حل مسأله‌ی بخش بحرانی فراهم کرده و برخی پیچیدگی‌های مشمول طراحی نرم‌افزاری را که به دنبال ارضای نیازمندی‌های انحصار متقابل، پیشروی و انتظار محدود است، روشن‌تر می‌سازد.

راه‌حل پترسن به دو پردازه محدود می‌شود که یکی پس از دیگری، بخش‌های بحرانی و باقیمانده‌ی خود را اجرا می‌کنند. پردازه‌ها با P_0 و P_1 شماره‌گذاری می‌شوند. برای راحتی، زمانی که یکی از پردازه‌ها با P_i نشان داده می‌شود، برای مشخص‌کردن پردازه‌ی دیگر از P_j استفاده می‌کنیم که j برابر $1 - i$ است.

راه‌حل پترسن به دو قلم‌داده‌ی مشترک در میان دو پردازه نیاز دارد:

```
int turn;
boolean flag[2];
```

متغیر turn نشان می‌دهد که نوبت کدام پردازه است که وارد بخش بحرانی خود شود. یعنی، اگر `turn == i` باشد، در این صورت پردازه‌ی P_i مجاز است که بخش بحرانی خود را اجرا کند. آرایه‌ی `flag` به این منظور استفاده می‌شود تا نشان دهد، آیا پردازه برای ورود به بخش بحرانی خود *آمادگی* دارد. برای مثال، اگر `flag[i]` برابر `true` باشد، P_i برای ورود به بخش بحرانی خود آماده است. اکنون با توضیح کاملی که از این ساختمان‌داده‌ها ارائه شد، آماده هستیم تا الگوریتم نشان داده شده در شکل ۶-۳ را توصیف کنیم.

پردازه‌ی P_i برای ورود به بخش بحرانی خود ابتدا `flag[i]` را `true` می‌کند، سپس `turn` را برابر `j` قرار می‌دهد تا با این کار، نوبت را به پردازه‌ی دیگر بدهد که خواهان ورود به بخش بحرانی خود می‌باشد. اگر هر دوی پردازه‌ها سعی داشته باشند که همزمان وارد شوند، متغیر `turn` تقریباً همزمان برابر مقدار `i` و `j` انتخاب خواهد شد. سرانجام، فقط یکی از این مقادیر ماندگار می‌شود؛ دیگری اتفاق هم بیفتد، بلافاصله رونویسی خواهد شد. مقدار نهایی `turn` تعیین می‌کند که کدام یک از دو پردازه مجاز است به بخش بحرانی خود وارد شود.

```
while (true) {
```

1. Peterson

```
        flag[i]= true;
        turn = j;
        while (flag[j] && turn == j);

        /* critical section */

        flag[i] = false;

        /* remainder section */
}
```

شکل ۶-۳ ساختار پردازه‌ی P_i در راه‌حل پترسن.

اکنون ثابت می‌کنیم که این راه‌حل درست است. لازم است که نشان دهیم:

۱- انحصار متقابل برقرار است.

۲- نیاز پیشروی ارضا می‌شود.

۳- نیاز انتظار محدود برآورده می‌شود.

برای اثبات ویژگی ۱، توجه داریم که P_i فقط در صورتی وارد بخش بحرانی خود می‌شود که `flag[j]== false` و یا `turn == i` باشد. همچنین توجه داشته باشید که اگر قرار باشد هر دو پردازه همزمان در حال اجرای بخش بحرانی خود باشند، در این صورت `flag[0] == flag[1] == true` خواهد بود. از این دو مشاهده چنین بر می‌آید که P_0 و P_1 نمی‌توانند تقریباً همزمان از حلقه‌ی `while` خود با موفقیت عبور کنند، چون مقدار `turn` می‌تواند ۰ و یا ۱ باشد، نه هر دو. از این رو، یکی از پردازه‌ها—بر فرض، P_j—بایستی دستور `while` را با موفقیت اجرا کرده باشد، در حالی که P_i مجبور است حداقل یک دستور اضافی دیگر نیز اجرا نماید ("`turn == j`"). با این وجود، در آن لحظه `flag[j]== true` و `turn == j` می‌باشد و این شرط مادامی که P_j در بخش بحرانی خود قرار دارد، ماندگار خواهد بود؛ در نتیجه، انحصار متقابل برقرار می‌شود.

برای اثبات ویژگی‌های ۲ و ۳، توجه داریم که پردازه‌ی P_i فقط در صورتی نمی‌تواند به بخش بحرانی خود وارد شود که در حلقه‌ی `while` با شرط `flag[j]== true` و `turn == j` مواجه شود؛ این حلقه تنها حلقه‌ی ممکن است. اگر P_j آمادگی ورود به بخش بحرانی را نداشته باشد، در این صورت `flag[j]== false` خواهد بود و P_i می‌تواند به بخش بحرانی خود وارد شود. اگر P_j، `flag[j]` را `true` کرده و در حال اجرای دستور `while` خود هم باشد، در این صورت `turn == i` و یا `turn == j` خواهد بود. اگر `turn == i` باشد، در این صورت P_i وارد بخش بحرانی خواهد شد. اگر `turn == j` باشد، در این صورت P_j وارد بخش بحرانی خواهد شد. با این وجود، همین که P_j از بخش بحرانی خود خارج می‌شود، `flag[j]` را `false` خواهد نمود که با این کار به پردازه‌ی P_i اجازه می‌دهد تا وارد بخش بحرانی خود شود. اگر P_j، `flag[j]` را دومرتبه `true` انتخاب کند، بایستی `turn` را برابر `i` انتخاب کند. بنابراین، از آنجایی که P_i، مقدار متغیر `turn` را در حال اجرای دستور `while` تغییر نمی‌دهد، حداکثر بعد از یکبار ورود P_j (انتظار محدود)، پردازه‌ی P_i به بخش بحرانی خود وارد خواهد شد (پیشروی).

همان‌گونه که در شروع این بخش عنوان شد، راه‌حل پترسن تضمینی نیست که در معماری‌های مدرن کامپیوتر کارگر باشد، به این دلیل اصلی که پردازنده‌ها و/یا کامپایلرها به‌منظور بهبود کارایی سیستم ممکن است ترتیب عملیات خواندن و نوشتن را که وابستگی ندارند تغییر دهند. این تجدید ترتیب در یک برنامه‌ی تک‌نخی مهم نیست، مادامی که صحت برنامه اهمیت دارد، چرا که مقادیر نهایی با چیزی که انتظار می‌رود سازگار هستند. (این موضوع شبیه به موجودی دسته‌چک است—ترتیب واقعی که عملیات دریافت و پرداخت انجام می‌شوند اهمیتی ندارد، چون موجودی نهایی همچنان یکسان خواهد ماند.) اما در یک برنامه‌ی چندنخی با داده‌های مشترک، تجدید ترتیب دستورالعمل‌ها ممکن است منجر به نتایج ناسازگار یا غیرمنتظره شود.

به‌عنوان یک مثال، داده‌های زیر را در نظر بگیرید که میان دو نخ مشترک هستند:

```
boolean flag = false;
int x = 0;
```

که نخ ۱ دستورات زیر را اجرا می‌کند

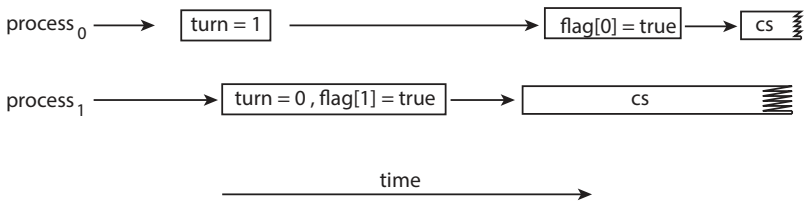
```
while (!flag);
print x;
```

و نخ ۲، دستورات زیر را اجرا می‌کند

```
x = 100;
flag = true;
```

البته، رفتار مورد انتظار این است که نخ ۱، مقدار ۱۰۰ را برای متغیر x خروجی نماید. با این وجود، چون هیچ‌گونه وابستگی داده‌ای میان متغیرهای flag و x وجود ندارد، این امکان هست که یک پردازنده ترتیب دستورالعمل‌های نخ ۲ را تغییر دهد طوری که flag قبل از دستور انتساب x = 100 برابر true مقداردهی شود. در این شرایط، این امکان وجود دارد که نخ ۱ برای متغیر x، خروجی 0 را نمایش دهد. با وضوح کمتر ممکن است پردازنده ترتیب دستورالعمل‌های صادر شده توسط نخ ۱ را تغییر داده و متغیر x را قبل از بارگذاری مقدار flag بارگذاری نماید. اگر این وضعیت پیش بیاید، نخ ۱ برای متغیر x خروجی 0 را خواهد داد حتی اگر ترتیب دستورالعمل‌های صادر شده توسط نخ ۲ تغییر نکرده باشند.

این موضوع چگونه راه‌حل پترسن را متأثر می‌سازد؟ در نظر بگیرید که چه اتفاقی می‌افتد در صورتی که ترتیب انتساب‌های دو دستورالعمل اولی که در بخش ورود راه‌حل پترسن در شکل ۶-۳ قرار دارند تغییر کند؛ این امکان هست که هر دو نخ، هم‌زمان در بخش‌های بحرانی خودشان، آن‌گونه که در شکل ۶-۴ نشان داده شده است، فعال باشند.



شکل ۶-۴ آثار تغییر ترتیب دستورالعمل در راه‌حل پترسن.

همان‌گونه که در بخش‌های بعدی خواهید دید، تنها راه حفظ انحصار متقابل، استفاده از ابزار همگامی مناسب است. بحث ما درباره‌ی این ابزار با پشتیبانی بدوی در سخت‌افزار شروع شده و با APIهای انتزاعی سطح بالای نرم‌افزاری در دسترس برای هر دوی توسعه‌دهندگان هسته و برنامه‌نویسان کاربردی ادامه پیدا می‌کند.

۶-۴ پشتیبانی سخت‌افزاری برای همگامی

ما یک راه‌حل نرم‌افزاری را برای مسئله‌ی بخش بحرانی مطرح کردیم. (ما از آن تحت عنوان راه‌حل ***نرم‌افزاری*** یاد می‌کنیم چون الگوریتم آن هیچ‌گونه پشتیبانی خاصی را از جانب سیستم‌عامل یا دستورالعمل‌های سخت‌افزاری خاصی برای تضمین انحصار متقابل لازم ندارد.) با این وجود، همان‌گونه که بحث شد، راه‌حل‌های نرم‌افزاری تضمینی نیست که در معماری‌های مدرن کامپیوتر کارگر باشند. در این بخش، سه دستورالعمل سخت‌افزاری را ارائه می‌کنیم که پشتیبانی حل مسئله‌ی بخش بحرانی را فراهم می‌کنند. این عملیات بدوی می‌توانند مستقیماً به‌عنوان ابزار همگامی استفاده شوند، یا می‌توانند برای شکل‌دهی بنیان چند راهکار همگامی انتزاعی استفاده شوند.

۶-۴-۱ سدهای حافظه

در بخش ۶-۳ دیدیم که یک سیستم ممکن است ترتیب دستورالعمل‌ها را تغییر دهد، سیاستی که می‌تواند به حالات داده‌ای نامطمئن منجر شود. چگونه یک معماری کامپیوتر تعیین می‌کند چه تضمین‌های حافظه برای یک برنامه‌ی کاربردی فراهم خواهد کرد به **مدل حافظه** معروف است. در حالت کلی، مدل حافظه در یکی از دو دسته‌ی زیر قرار می‌گیرد:

۱- **ترتیب قوی**[1]، که در آن تغییرات حافظه بر روی یک پردازنده بلافاصله برای سایر پردازنده‌ها قابل رؤیت است.

۲- **ترتیب ضعیف**[2]، که در آن، تغییرات حافظه بر روی یک پردازنده ممکن است بلافاصله برای سایر پردازنده‌ها قابل رؤیت نباشد.

مدل‌های حافظه بر اساس نوع پردازنده فرق دارند، لذا توسعه‌دهندگان هسته نمی‌توانند هرگونه فرضی در رابطه با قابلیت دید تغییرات حافظه بر روی یک چندپردازنده‌ی حافظه‌ی

1. Strongly ordered
2. Weakly ordered